Watson Platform

Engineering Proposal & Statement of Work

Prepared for: David — WhatCost.co.uk

Prepared by: Noventis Digital

Date: 7 April 2026

Version: 1.0

Status: Draft for review

Noventis Digital for **WhatCost.co.uk**

1. Executive Summary

Watson is a modern lead operations platform designed to replace the legacy FLG/WhatPrice stack. The product vision is clear: one platform for lead intake, qualification, referral routing, partner management, billing, returns, and reporting, with a proper API layer underneath.

Significant progress has been made. The admin UI is well advanced, the database schema is solid, the referral engine logic is substantially built, and the domain model captures the right commercial concepts. The platform's architecture (NestJS API, Next.js frontend, Prisma/Postgres, shared domain package) is modern and appropriate for the scale.

What remains is engineering depth: making the persistence layer real, wiring up operational execution (referral dispatch, billing, webhooks), hardening security, building the installer dashboard, establishing deployment infrastructure, and later integrating with advertising platforms and AI capabilities.

> **Core thesis:** The product shape is right. The admin surface demonstrates the operating model. The biggest remaining steps are persistence, API foundations, security, and deployment — not more UI.

How We Work

> **AI-assisted engineering:** Noventis Digital uses AI coding tools as a core part of the development workflow. This means faster delivery, consistent code quality, and more time spent on architecture and review rather than typing. The hour estimates in this proposal reflect this working method — they represent real consultant time including AI-assisted code generation, human review, testing, integration, and decision-making. We're transparent about this because it's how modern software gets built efficiently.

Watson itself was largely built using AI-assisted development, which is why the breadth of UI and scaffolding is impressive. What AI tools are less good at — and where this engagement focuses — is the depth work: security hardening, real persistence, operational wiring, deployment, and the careful integration work that makes a demo into a product.

2. Current State Assessment

Technology Stack

Component	Technology
Monorepo	PNPM workspaces + Turbo
API	NestJS 11 (TypeScript) — port 4000
Web	Next.js 15 with React 19 (TypeScript) — port 3000
Database	PostgreSQL 16 + Prisma 6 ORM
Domain Logic	Shared @watson/domain package
Billing	Xero integration (OAuth, mock/live modes)

What Exists

Area	Status	Detail
Admin UI	~80%	~50 routes. Leads, partners, lead groups, workflows, billing, returns, distribution, reports, users.
Database schema	DONE	16+ tables: leads, partners, coverage, billing, workflows, rules, field catalog. Well-normalised.
Referral engine logic	~70%	Release planning, coverage matching, commercial calculations, provider resolution, retry logic. Execution not wired.
API surface	~45%	~55 endpoints. Config CRUD strong. No inbound lead submission, no M2M auth, no outbound webhooks.
Workflow definitions	~65%	UI for creation/editing done. No runtime execution engine.
Billing / Xero	~40%	Mirror structure, OAuth token handling. No real invoice generation or sync lifecycle.
Partner portal	~15%	Dashboard and billing page only. No lead management, coverage, settings, or self-service.
Security	~10%	JWT generation exists. No auth guards on endpoints. No rate limiting, CSRF, or input sanitisation.
Deployment	0%	No Docker, CI/CD, environment config, or hosting setup.

Ad integration	0%	Not started.
AI voice / chatbot	0%	Not started.

Key Technical Risks

> **Dual-runtime problem:** Every API service tries PostgreSQL first, then falls back to JSON flat files. This means the app appears to work without a real database, masking issues. All features built on this pattern will need rework. This is the single largest piece of technical debt.

- **No tracked migrations** — Prisma schema exists but migration history does not. Any production schema change without tracked migrations is destructive.
- **No auth enforcement** — All 55 API endpoints are publicly accessible. No guards, no role checks.
- **No async processing** — No message queue for delivery retries, billing runs, or webhook dispatch.
- **No observability** — No error tracking, structured logging, or health dashboards.

3. Scope of Work

The work is structured in six phases, ordered by dependency and business value. Each phase delivers a tangible, testable increment.

Phase 1: Foundations — Make It Real

Eliminate the dual-runtime, establish database-first operations, secure the API, and create a deployable artifact.

Task	Detail
Stand up PostgreSQL	Docker Compose for local dev. Connection verified. Prisma migrations generating tracked history.
Remove JSON fallback	Remove all fallback paths from services. JSON files become seed data only, not runtime storage.
Expand seed data	Migrate demo scenarios currently only in JSON into proper seed scripts covering partners, leads, billing, workflows.
API auth guards	JWT auth guards on all non-public endpoints. Role-based access (internal vs partner). Rate limiting.
Input validation & security	Sanitisation beyond basic DTOs. CSRF protection. Secure headers. Dependency audit.
Deployment pipeline	Dockerfiles for API + Web. Docker Compose for full stack. Basic CI (lint, type-check, test). Environment variable management.
Migration baseline	Generate initial migration from current schema. Establish migration workflow for all future changes.

> **Outcome:** Watson runs on a real database, endpoints are secured, and it can be deployed to a server. This unblocks all subsequent phases.

Phase 2: Referral Execution

Wire up the referral engine so leads actually get dispatched to partners. The domain logic exists — the execution layer doesn't.

Task	Detail
Outbound HTTP delivery	POST/GET to partner endpoints using delivery templates in PartnerSettings. XML/JSON/FORM payloads.
Outbound email delivery	Brevo integration (adapter exists, needs actual send wiring).
Manual-accept flow	Offer window with configurable timeout, batch exposure, next-partner fallback.
Delivery logging	Per-attempt records in DB. Request/response capture. Status tracking.
Partner callback endpoint	Inbound endpoint for partners to POST referral status updates (accepted, converted, returned).
Retry & escalation	Failed delivery retry with backoff. Escalation to marketplace or next partner.
Async job queue	Background processing for delivery dispatch, retries, timeouts (BullMQ/Redis).

> **Outcome:** A lead enters Watson, gets classified, gets dispatched to the right partner via their configured channel, and the partner can respond. This is the operational core.

Phase 3: API Completion

Build the API surface needed for external integrations and the installer dashboard.

Task	Detail
Inbound lead submission	`POST /api/leads` with validation, deduplication, lead-type matching, auto-routing.
M2M authentication	API keys per client. Signed requests. Scoped permissions.
Partner-scoped endpoints	Accept/decline referral, update lead status, request return, view assigned leads — all scoped to authenticated partner.
Outbound webhooks	Event dispatch on lead status change, referral outcome, return decision. Configurable per partner.
OpenAPI specification	Auto-generated from NestJS decorators. Published documentation.
GDPR / data protection	Data retention policies. Right-to-erasure endpoint. Audit logging for PII access.

> **Outcome:** Watson has a complete, documented, secured API that external systems and the installer dashboard can consume.

Phase 4: Installer Dashboard

Build the partner-facing portal on top of Watson's API. Matches current installer-dashboard capabilities.

Task	Detail
Lead management	My leads list, lead detail, report issues, buy leads (marketplace).
Coverage self-service	Region/postcode selection. Adapt existing location-targeting-matrix component for partner use.
Account settings	Company details, delivery preferences, alert configuration, notification preferences.
Invoicing & payments	View invoices, payment history, spending summary.
Company profile	Business listing, accreditations, work types, service descriptions.
Lead alerts	Configurable notifications for matching leads (email, push).

> **Outcome:** Partners have a self-service portal matching current installer-dashboard capabilities, consuming the same Watson API as admin.

Phase 5: Billing Completion

Close the billing loop from referral outcome through to Xero invoice.

Task	Detail
Invoice generation	From billing run preview to actual invoice rows in DB with line items.
Xero sync lifecycle	Create invoice, attach schedule CSV, handle sync failures, retry with state tracking.
OAuth token management	Refresh flow hardening, expiry handling, health reporting.
Returns financial flow	Return approval triggers credit note/refund, ledger entry, Xero sync.
Billing policies	Weekly batched and individual invoicing modes. Configurable per partner.
Statement generation	Partner-facing statements with transaction history, CSV/PDF export.

> **Outcome:** Referral outcomes flow through to real invoices in Xero. Partners see accurate billing. Returns have financial consequences. Xero remains the accounting system of record.

Phase 6: Ad Integration & AI

Tighter advertising platform integration and AI-assisted lead qualification.

Task	Detail
Google Ads integration	Offline conversion upload, ROAS reporting, bid signal feedback via Google Ads API.
Microsoft Ads integration	Offline conversion tracking, campaign performance sync.
Meta Ads integration	Conversions API for offline event upload, custom audience sync.
WhatsApp / SMS chatbot	Lead qualification via conversational flow. WhatsApp Business API + Twilio/MessageBird for SMS.
AI voice bot	Outbound calling for leads unresponsive to text channels. Integration with voice AI platform (Vapi, Bland.ai, or similar) calling Watson's API.
Transactional email	Integration with email service (Brevo, Postmark) for system notifications, referral confirmations, partner alerts.

Outcome: Advertising spend feeds back to outcomes. Leads that don't qualify via digital channels get called. Watson becomes the operational hub connecting ad platforms, communication channels, and partner operations.

4. Estimated Effort by Phase

All estimates reflect AI-assisted development: code generation, human review, testing, integration, and architectural decision-making. Ranges account for complexity discovered during implementation.

#	Phase / Task	Hours
Phase 1: Foundations — Make It Real		
	PostgreSQL setup + Docker Compose	3 - 4
	Remove JSON fallback from all services	8 - 10
	Expand seed data & migration baseline	4 - 5
	API auth guards & role enforcement	5 - 7
	Input validation, CSRF, secure headers	4 - 5
	Dockerfiles, CI pipeline, environment config	6 - 8
	Testing & verification	4 - 5
	Phase 1 Subtotal	**34 - 44**
Phase 2: Referral Execution		
	Outbound HTTP delivery (POST/GET, payload templates)	8 - 10
	Outbound email delivery (Brevo)	3 - 5
	Manual-accept flow & offer timeout	7 - 9
	Delivery logging & status tracking	4 - 6
	Partner callback endpoint	3 - 5
	Retry, backoff & escalation logic	5 - 7
	Async job queue setup (BullMQ / Redis)	6 - 8
	Testing & verification	5 - 6
	Phase 2 Subtotal	**41 - 56**
Phase 3: API Completion		
	Inbound lead submission endpoint	6 - 8
	M2M authentication (API keys, signing)	5 - 7

	Partner-scoped API endpoints	7 - 10
	Outbound webhook system	5 - 7
	OpenAPI spec generation & docs	2 - 3
	GDPR / data protection	4 - 6
	Testing & verification	4 - 5
	Phase 3 Subtotal	**33 - 46**
Phase 4: Installer Dashboard		
	Lead management (list, detail, report, buy)	8 - 12
	Coverage self-service	5 - 7
	Account settings & company profile	5 - 8
	Invoicing & payment history	4 - 6
	Lead alerts & notifications	4 - 6
	Testing & verification	4 - 5
	Phase 4 Subtotal	**30 - 44**
Phase 5: Billing Completion		
	Invoice generation & line items	6 - 8
	Xero sync lifecycle & retry	8 - 10
	OAuth token management hardening	3 - 4
	Returns financial flow (credits, refunds)	5 - 7
	Billing policies & statement generation	4 - 6
	Testing & verification	3 - 4
	Phase 5 Subtotal	**29 - 39**
Phase 6: Ad Integration & AI		
	Google Ads offline conversion & ROAS	8 - 10
	Microsoft Ads integration	5 - 7
	Meta Conversions API	5 - 8
	WhatsApp / SMS chatbot integration	10 - 14

	AI voice bot integration	10 - 14
	Transactional email platform	3 - 5
	Testing & verification	5 - 7
	Phase 6 Subtotal	**46 - 65**
	Total Estimated Effort	**213 - 294**

Phase Summary

Phase	Focus	Hours	Dependency
1	Foundations	34 - 44	None — start here
2	Referral Execution	41 - 56	Phase 1
3	API Completion	33 - 46	Phase 1
4	Installer Dashboard	30 - 44	Phase 3
5	Billing Completion	29 - 39	Phase 2
6	Ad Integration & AI	46 - 65	Phases 2 + 3

About these estimates: These hours reflect AI-assisted development — code generation, automated testing, and intelligent scaffolding are used throughout. The consultant's time is spent on architecture, review, integration, security hardening, and the judgment calls that AI tools don't make well on their own. This is a transparent reflection of how modern software engineering works: AI handles volume, humans handle correctness.

Parallelism: Phases 2 and 3 can run concurrently after Phase 1. Phases 4 and 5 can overlap. Phase 6 begins once the API layer is stable. Realistic delivery: Phases 1-4 in 8-12 weeks, full scope in 14-20 weeks.

5. Statement of Work

5.1 Objective

Take the Watson platform from its current prototype/demo state to a production-ready lead operations system with secured API, real database persistence, operational referral dispatch, installer-facing dashboard, billing integration, and deployment infrastructure.

5.2 Approach

- **AI-assisted development** — AI coding tools are used for code generation, testing, and analysis. The consultant provides architecture, security review, integration work, and quality assurance. This is disclosed transparently.
- **Phased delivery** — Each phase produces a deployable, testable increment. No phase depends on all others completing first.
- **Existing code respected** — The current UI, schema, and domain logic are preserved and built upon, not rewritten.
- **External platforms for external concerns** — Xero for accounting, Brevo/Postmark for email, Twilio for SMS, dedicated platforms for chatbots and voice. Watson orchestrates, it doesn't replace specialised tools.
- **API-first** — The installer dashboard and all external integrations consume Watson's API. No direct database access from frontend or partner systems.

5.3 Deliverables per Phase

Phase	Deliverables
1	Postgres-backed runtime. Auth-protected API. Dockerised deployment. CI pipeline. Migration baseline.
2	Working referral dispatch (HTTP + email). Manual-accept flow. Delivery logging. Callback handling. Job queue.
3	Lead submission API. M2M auth. Partner-scoped endpoints. Webhooks. OpenAPI docs. GDPR compliance.
4	Installer dashboard with leads, coverage, settings, billing, profile, alerts.
5	Invoice generation. Xero sync lifecycle. Returns financial flow. Billing policies. Statements.
6	Google/Microsoft/Meta ad integration. WhatsApp/SMS chatbot. AI voice bot. Transactional email.

5.4 Assumptions

- Access to Watson codebase, Xero sandbox, and relevant third-party API credentials will be provided.

- David or designated stakeholder available for clarification and review at phase boundaries.
- Hosting infrastructure (VPS, cloud provider) to be agreed before Phase 1 deployment task.
- Ad platform and AI voice provider accounts to be set up before Phase 6.
- Existing admin UI accepted as-is — cosmetic refinements are out of scope unless specifically requested.

5.5 Out of Scope

- Rewriting existing UI/UX (admin pages, component library).
- Building a custom accounting system (Xero is the system of record).
- Mobile native apps.
- Data migration from legacy FLG/WhatPrice/Sherlock (separate engagement if required).
- Ongoing maintenance and support post-delivery (can be quoted separately).

5.6 Working Method

- Work delivered in focused blocks aligned to phases.
- Code committed to Watson repository with clear commit messages.
- AI-assisted development used throughout — transparent and documented.
- Review checkpoints at each phase boundary.
- Consultant advises on priorities and technical decisions; David retains final product direction.

6. Recommendations

Start with Phase 1 immediately

Everything else builds on a real database and secured API. Every feature added before this is done will need partial rework. This is the highest-leverage work.

Don't add more admin UI

The admin surface is already at ~80%. Resist the temptation to polish pages before the backend is real. The UI will naturally evolve as real data flows through it.

Lean on external platforms

David's instinct here is right. Xero for invoicing, Brevo/Postmark for email, Twilio for SMS, WhatsApp Business API for chat, a voice AI platform for calling. Watson should orchestrate, not replicate.

Deploy early, not late

Getting Watson running on a real server in Phase 1 (even with limited features) surfaces problems that only appear in production: DNS, SSL, environment variables, database connectivity, performance. Better to find these on day one than month six.

Consider data migration separately

If Watson needs to import historical data from FLG/Sherlock, that's a meaningful piece of work that should be scoped independently once the Watson data model is stable.

This document was prepared by Noventis Digital based on a thorough technical audit of the Watson codebase conducted on 7 April 2026. Estimates are good-faith projections based on AI-assisted development and may be refined as work progresses. All code and deliverables remain the property of WhatCost.co.uk.